UNIVERSIDAD CES · BASES DE DATOS

Modelo de datos y diccionario

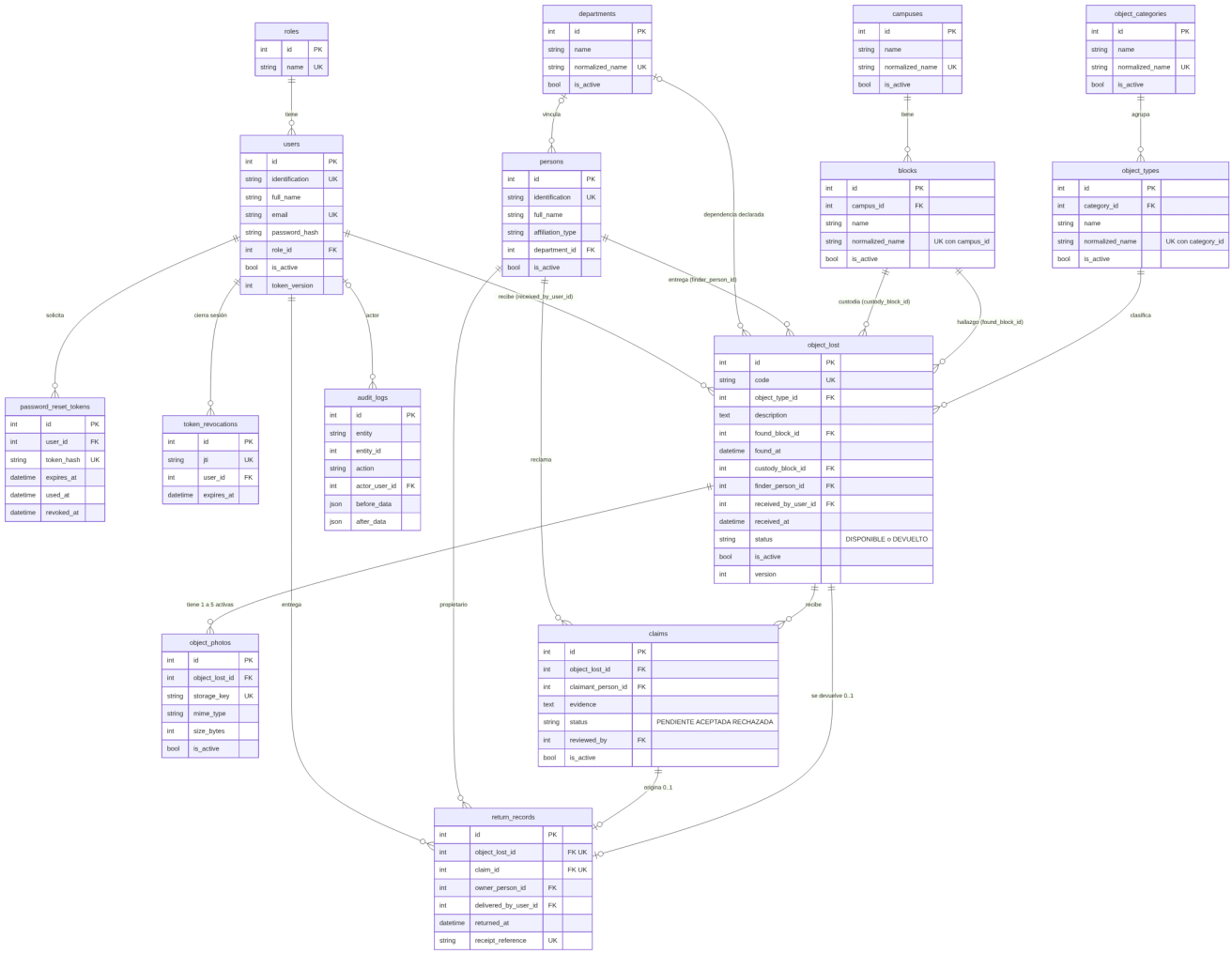
Diagrama entidad-relación, normalización, historial y diccionario de datos

---

Proyecto final: Gestión de objetos perdidos
Juan Alejandro Chaux Perdomo · Noviembre de 2026

2. Modelo de datos

Motor: **PostgreSQL 16** (cualquier versión 14+ sirve). Las tablas se crean con **Flask-Migrate**: 7 migraciones generadas con `flask db migrate` (una por grupo de cambios, como en clase) y 1 revisión escrita a mano para lo que Alembic no detecta (triggers).



El diagrama muestra las columnas principales; el detalle completo (tipos, nulabilidad, FK, CHECK e índices) está en `03_diccionario_datos.md`. La fuente del diagrama es `img/diagrama_er.mmd` (Mermaid). En DBeaver: clic derecho en el esquema `public` → *View Diagram*.

Migraciones

Revisión	Mensaje	Qué crea
`2ce4759b8957`	create roles and users tables	`roles`, `users`
`8cc5e752e915`	create password reset and token revocation tables	`password_reset_tokens`, `token_revocations`
`e9355e9dc03f`	create catalog tables	`campuses`, `blocks`, `departments`, `object_categories`, `object_types`
`bfabf501638e`	create persons table	`persons`
`15059d970a87`	create object lost and object photos tables	`object_lost`, `object_photos`

Revisión	Mensaje	Qué crea
`9db9c7695304`	create claims and return records tables	`claims`, `return_records` (incluye el índice único parcial)
`f357c059a7eb`	create audit logs table	`audit_logs`
`8d6746a80648`	add triggers that block delete and update of history	funciones `prevent_delete()`, `prevent_update()` y sus triggers (manual)

Relaciones y cardinalidades

Relación	Cardinalidad	FK
Role → User	1:N	`users.role_id`
Campus → Block	1:N	`blocks.campus_id`
ObjectCategory → ObjectType	1:N	`object_types.category_id`
Department → Person	1:N (opcional)	`persons.department_id`
ObjectType → ObjectLost	1:N	`object_lost.object_type_id`
Block → ObjectLost	1:N dos veces (hallazgo y custodia)	`found_block_id`, `custody_block_id`
Person → ObjectLost	1:N (quien entrega)	`finder_person_id`
User → ObjectLost	1:N (quien recibe)	`received_by_user_id`
ObjectLost → ObjectPhoto	1:N (1 a 5 activas)	`object_photos.object_lost_id`
ObjectLost → Claim	1:N	`claims.object_lost_id`
Person → Claim	1:N (reclamante)	`claims.claimant_person_id`
ObjectLost → ReturnRecord	1:0..1	`return_records.object_lost_id` **UNIQUE**
Claim → ReturnRecord	1:0..1	`return_records.claim_id` **UNIQUE**
User → AuditLog	1:N (actor)	`audit_logs.actor_user_id`

Todas las FK son `ON DELETE RESTRICT`: no hay borrado en cascada.

Person y User son entidades distintas. User inicia sesión y tiene rol; Person es alguien externo que entrega o reclama sin cuenta.

ObjectType vs ObjectLost. ObjectType es el catálogo («Celular»); ObjectLost es un objeto físico concreto. Dos celulares son dos filas de `object_lost` con su código, fotos y responsables. El tipo nunca se escribe como texto en `object_lost`: se guarda `object_type_id`.

Restricciones que protegen la integridad

Restricción	Para qué
`UNIQUE(normalized_name)` en sedes, dependencias y categorías; `UNIQUE(campus_id, normalized_name)` en bloques; `UNIQUE(category_id, normalized_name)` en tipos	Evitar duplicados aunque cambien mayúsculas, tildes o espacios, incluso con dos peticiones simultáneas.
`CHECK status IN ('DISPONIBLE','DEVUELTO')`, `CHECK version >= 1`, `CHECK found_at <= received_at`	Estados válidos y fechas coherentes en `object_lost`.
`CHECK mime_type IN (...)` y `CHECK size_bytes > 0 AND <= 5242880`	Metadata de fotos válida.
`CHECK affiliation_type IN (...)`	Tipos de vinculación válidos.

Restricción	Para qué
`CHECK status IN ('PENDIENTE','ACEPTADA','RECHAZADA')`	Estados de reclamación.
Índice único parcial `uq_claims_one_accepted_per_object WHERE status = 'ACEPTADA'`	A lo sumo una reclamación aceptada por objeto (las pendientes y rechazadas pueden ser varias).
`UNIQUE return_records.object_lost_id` y `claim_id`	Una sola devolución por objeto y por reclamación.
`CHECK owner_confirmation = TRUE`	No existe devolución sin confirmación del propietario.
Triggers `prevent_delete` / `prevent_update`	Nada se borra; auditoría y devoluciones son de solo lectura.

Índices para los filtros

`object_lost`: `code` (único), `object_type_id`, `found_block_id`, `custody_block_id`, `finder_person_id`, `received_at`, `status`, `is_active` y el compuesto `(status, is_active)` que usa el inventario. También índices en todas las FK de búsqueda (`blocks.campus_id`, `object_types.category_id`, `claims.object_lost_id`, `claims.status`, `audit_logs (entity, entity_id)`, `audit_logs.occurred_at`, etc.).

Normalización

El modelo está en **tercera forma normal**:

- 1FN: cada columna guarda un solo valor; las fotos (varias por objeto) están en su propia tabla en vez de columnas `foto1…foto5`; un objeto es una unidad física (no hay columna «cantidad»).
- 2FN: todas las tablas tienen clave primaria simple (`id`), así que no hay dependencias parciales.
- 3FN: no hay dependencias transitivas guardadas. Ejemplo: la sede del hallazgo **no** se guarda en `object_lost`; se obtiene de `found_block_id → blocks.campus_id`. El nombre del tipo y su categoría tampoco se repiten en el objeto.

Desnormalizaciones conscientes: `normalized_name` (derivado de `name`) existe para que la base pueda exigir UNIQUE sobre él; `return_records.owner_person_id` repite al reclamante de la reclamación para que el histórico de la devolución sea autocontenido.

Historial

Cada creación, edición, inactivación, reactivación, decisión y devolución inserta una fila en `audit_logs` **en la misma transacción** del cambio, con `before_data` y `after_data` (solo los campos que cambiaron) en JSON. Nunca guarda contraseñas, hashes de tokens ni archivos. La tabla no acepta UPDATE ni DELETE (trigger). Además, los registros conservan `created_by`, `updated_by`, `inactivated_at`, `inactivated_by` e `inactivation_reason`.

Diccionario de datos

Generado desde la base PostgreSQL después de `flask db upgrade` (8 migraciones). Convenciones: PK = clave primaria, FK = clave foránea (todas con `ON DELETE RESTRICT`, sin cascadas), UK = único. Todas las fechas son `timestamp with time zone` y se guardan en UTC; la interfaz las muestra en hora de Bogotá. Todas las tablas tienen triggers `trg_<tabla>_no_delete` y `trg_<tabla>_no_truncate`; `audit_logs` y `return_records` además `trg_<tabla>_no_update`.

roles

Roles del sistema (ADMIN, OPERADOR, CONSULTA). Un rol tiene muchos usuarios.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
name	varchar(30)	no	UK	
description	varchar(200)	sí		
created_at	timestamptz	no		

Índices: `roles_name_key`

users

Cuentas que inician sesión. El rol se lee de aquí en cada petición; nunca se expone password_hash.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
identification	varchar(50)	no	UK	
full_name	varchar(150)	no		
email	varchar(200)	no	UK	
password_hash	varchar(255)	no		
role_id	integer	no	FK → roles.id (RESTRICT)	
is_active	boolean	no		
token_version	integer	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

Índices: `ix_users_role_id`; `users_email_key`; `users_identification_key`

password_reset_tokens

Tokens de recuperación: solo el hash SHA-256, vencen a los 15 minutos, un solo uso, nunca se borran.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
user_id	integer	no	FK → users.id (RESTRICT)	
token_hash	varchar(64)	no	UK	
created_at	timestamptz	no		
expires_at	timestamptz	no		
used_at	timestamptz	sí		
revoked_at	timestamptz	sí		

Índices: `ix_password_reset_tokens_user_id`; `password_reset_tokens_token_hash_key`

token_revocations

JWT revocados con logout (por jti).

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
jti	varchar(64)	no	UK	
user_id	integer	no	FK → users.id (RESTRICT)	
expires_at	timestamptz	no		
revoked_at	timestamptz	no		

Índices: `ix_token_revocations_user_id`; `token_revocations_jti_key`

campuses

Sedes. Nombre único normalizado.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
name	varchar(100)	no		
normalized_name	varchar(100)	no	UK	
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

Índices: `uq_campuses_normalized_name`

blocks

Bloques de una sede. Nombre único dentro de la sede.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
campus_id	integer	no	FK → campuses.id (RESTRICT)	
name	varchar(100)	no		
normalized_name	varchar(100)	no		
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

Únicos compuestos: `uq_blocks_campus_normalized_name` (campus_id, normalized_name) **Índices:** `ix_blocks_campus_id`; `uq_blocks_campus_normalized_name`

departments

Dependencias (facultades, oficinas).

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
name	varchar(150)	no		
normalized_name	varchar(150)	no	UK	
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

Índices: `uq_departments_normalized_name`

object_categories

Categorías de tipos de objeto.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
name	varchar(100)	no		
normalized_name	varchar(100)	no	UK	
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

Índices: `uq_object_categories_normalized_name`

object_types

Tipos reutilizables del catálogo (no son objetos físicos). Nombre único dentro de la categoría.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
category_id	integer	no	FK → object_categories.id (RESTRICT)	
name	varchar(100)	no		
normalized_name	varchar(100)	no		
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

Únicos compuestos: `uq_object_types_category_normalized_name` (category_id, normalized_name)
Índices: `ix_object_types_category_id`; `uq_object_types_category_normalized_name`

persons

Personas que entregan o reclaman sin tener cuenta.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
identification	varchar(30)	no	UK	
full_name	varchar(150)	no		
affiliation_type	varchar(20)	no		

Columna	Tipo	Nulo	Llaves	Default
department_id	integer	sí	FK → departments.id (RESTRICT)	
email	varchar(200)	sí		
phone	varchar(30)	sí		
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	sí	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

CHECK: `ck_persons_affiliation_type`: `CHECK (((affiliation_type)::text = ANY ((ARRAY['ESTUDIANTE'::character varying, 'EMPLEADO'::character varying, 'CONTRATISTA'::character varying, 'PROVEEDOR'::character varying, 'VISITANTE'::character varying])::text[])))` **Índices:** `ix_persons_department_id`; `persons_identification_key`

object_lost

Cada objeto físico recibido (una unidad por registro).

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
code	varchar(20)	no	UK	
object_type_id	integer	no	FK → object_types.id (RESTRICT)	
description	text	no		
found_block_id	integer	no	FK → blocks.id (RESTRICT)	
found_location	varchar(255)	no		
found_at	timestamptz	no		
custody_block_id	integer	no	FK → blocks.id (RESTRICT)	
custody_location	varchar(255)	no		
finder_person_id	integer	no	FK → persons.id (RESTRICT)	
finder_department_id	integer	sí	FK → departments.id (RESTRICT)	
received_by_user_id	integer	no	FK → users.id (RESTRICT)	
received_at	timestamptz	no		
status	varchar(20)	no		
is_active	boolean	no		
version	integer	no		
correction_reason	text	sí		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	no	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	

Columna	Tipo	Nulo	Llaves	Default
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

CHECK: `ck_object_lost_status`: `CHECK (((status)::text = ANY ((ARRAY['DISPONIBLE'::character varying, 'DEVUELTO'::character varying])::text[])))`; `ck_object_lost_found_before_received`: `CHECK ((found_at <= received_at))`; `ck_object_lost_version`: `CHECK ((version >= 1))` **Índices:** `ix_object_lost_custody_block_id`; `ix_object_lost_finder_person_id`; `ix_object_lost_found_block_id`; `ix_object_lost_is_active`; `ix_object_lost_object_type_id`; `ix_object_lost_received_at`; `ix_object_lost_status`; `ix_object_lost_status_is_active`; `object_lost_code_key`

object_photos

Metadata de las fotos; el archivo está en disco (UPLOAD_FOLDER).

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
object_lost_id	integer	no	FK → object_lost.id (RESTRICT)	
storage_key	varchar(100)	no	UK	
mime_type	varchar(20)	no		
size_bytes	integer	no		
uploaded_by_user_id	integer	no	FK → users.id (RESTRICT)	
uploaded_at	timestamptz	no		
is_active	boolean	no		
updated_at	timestamptz	no		
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

CHECK: `ck_object_photos_mime_type`: `CHECK (((mime_type)::text = ANY ((ARRAY['image/jpeg'::character varying, 'image/png'::character varying])::text[])))`; `ck_object_photos_size`: `CHECK (((size_bytes > 0) AND (size_bytes <= 5242880)))` **Índices:** `ix_object_photos_object_lost_id`; `object_photos_storage_key_key`

claims

Reclamaciones de propiedad y su decisión.

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
object_lost_id	integer	no	FK → object_lost.id (RESTRICT)	
claimant_person_id	integer	no	FK → persons.id (RESTRICT)	
evidence	text	no		

Columna	Tipo	Nulo	Llaves	Default
identity_verified	boolean	no		
verification_notes	text	sí		
status	varchar(20)	no		
reviewed_by	integer	sí	FK → users.id (RESTRICT)	
reviewed_at	timestamptz	sí		
decision_reason	text	sí		
correction_reason	text	sí		
is_active	boolean	no		
created_at	timestamptz	no		
updated_at	timestamptz	no		
created_by	integer	no	FK → users.id (RESTRICT)	
updated_by	integer	sí	FK → users.id (RESTRICT)	
inactivated_at	timestamptz	sí		
inactivated_by	integer	sí	FK → users.id (RESTRICT)	
inactivation_reason	text	sí		

CHECK: `ck_claims_status`: `CHECK (((status)::text = ANY ((ARRAY['PENDIENTE'::character varying, 'ACEPTADA'::character varying, 'RECHAZADA'::character varying])::text[])))` **Índices:** `ix_claims_claimant_person_id`; `ix_claims_object_lost_id`; `ix_claims_status`; `uq_claims_one_accepted_per_object` (único parcial: `((status)::text = 'ACEPTADA'::text)`)

return_records

Histórico de devoluciones (solo lectura: un trigger impide UPDATE y DELETE).

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
object_lost_id	integer	no	UK; FK → object_lost.id (RESTRICT)	
claim_id	integer	no	UK; FK → claims.id (RESTRICT)	
owner_person_id	integer	no	FK → persons.id (RESTRICT)	
delivered_by_user_id	integer	no	FK → users.id (RESTRICT)	
returned_at	timestamptz	no		
receipt_reference	varchar(30)	no	UK	
receipt_method	varchar(30)	no		
owner_confirmation	boolean	no		
verification_summary	text	no		
notes	text	sí		

CHECK: `ck_return_records_owner_confirmation`: `CHECK ((owner_confirmation = true))` **Índices:** `ix_return_records_owner_person_id`; `return_records_claim_id_key`; `return_records_object_lost_id_key`; `return_records_receipt_reference_key`

audit_logs

Historial de cambios con antes/después (solo inserción).

Columna	Tipo	Nulo	Llaves	Default
id	integer	no	PK	secuencia (serial)
entity	varchar(50)	no		
entity_id	integer	no		
action	varchar(50)	no		
actor_user_id	integer	sí	FK → users.id (RESTRICT)	
occurred_at	timestamptz	no		
reason	text	sí		
before_data	json	sí		
after_data	json	sí		

Índices: `ix_audit_logs_actor_user_id`; `ix_audit_logs_entity_entity_id`; `ix_audit_logs_occurred_at`